SOLUTION ARCHITECT · INTERVIEW PREPARATION

System Design Interview Preparation — American Express Solution Architect

Interview: Wednesday, November 4, 2026 **Goal:** Develop broad coverage of system-design patterns with extra depth in payments and enterprise architecture. **Learning style:** Worked designs, explicit trade-offs, brief concepts at point of use. **No quizzes, mock interviews, timed blocks, or assessments.**

1. Overview

Core strategy

Study **73 system-design problems across 8 modules**, but do **not** memorize 73 isolated diagrams. Understand the reusable architectural decisions underneath them. Cover the breadth first; then deepen the problems that combine the most essential patterns.

1. **Decision-first:** Start with a real requirement, then derive each component from it.
2. **One baseline before complexity:** Begin with the simplest acceptable design, and add components only to address a named constraint.
3. **Explain choices:** For every database, protocol, queue, cache, and deployment choice, compare plausible alternatives.
4. **Use financial examples as an anchor:** Payments, ledgers, fraud, reconciliation, security and auditability expose important correctness requirements.
5. **Think like a Solution Architect:** Include existing systems, integration contracts, organizational ownership, cost, migration, governance and operational readiness.
6. **No assessments:** This is a guided reference and explanation plan, not an examination program.

Success criteria

- Recognize core architectural patterns when a new problem is described.
- Explain **why** a design fits its constraints and **when** it would stop fitting.
- Describe data flow, state ownership, consistency, failure recovery, and security boundaries clearly.
- Reuse solved patterns across domains without forcing the same technology into every design.

Scope and priorities

Track	Coverage	Treatment
Architecture fundamentals	20 connected concepts	Explain in context, before or during designs
Payments / financial systems	16 case studies	Highest domain depth
Scalable applications	12 case studies	Broad reusable patterns
Distributed transactions / workflows	10 case studies	Consistency and failure depth
Data and streaming	9 case studies	Batch/stream/data ownership
Reliability and cloud	9 case studies	High-availability and recovery
Security and enterprise	9 case studies	Solution Architect lens
Advanced systems	8 case studies	Breadth/extensions
Total design cases	**73**	**16 recommended full deep dives**

Important: This plan is tailored to the Solution Architect role and financial-services domain. It is **not** a claim that American Express will ask these exact questions.

2. Method — How Each Design Is Explained

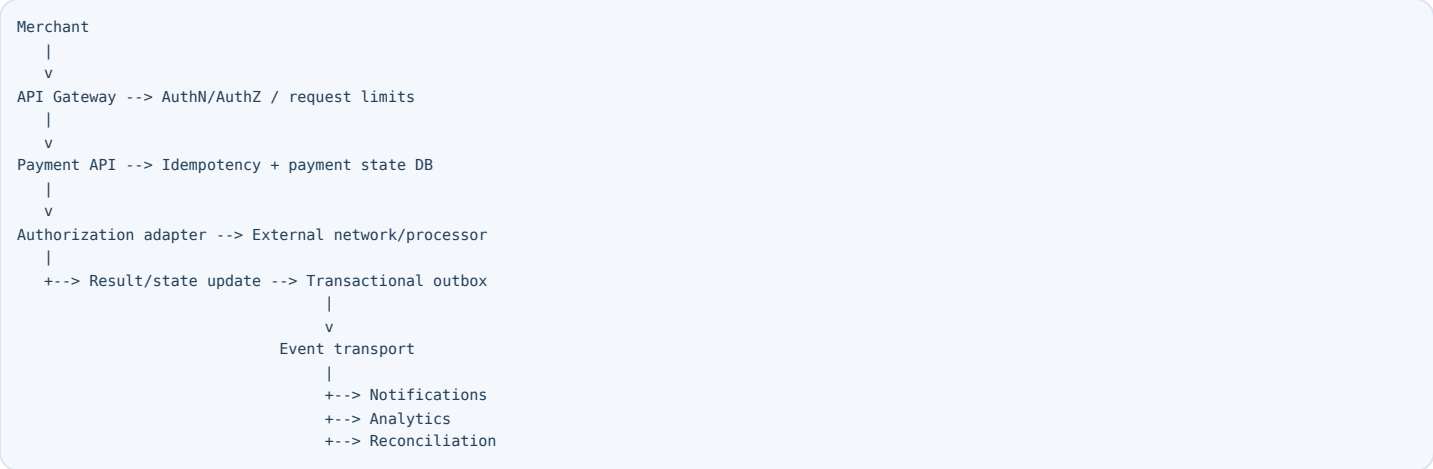
Use this structure consistently; increase or decrease detail according to the case.

Step	What we cover	Why it comes now
1. Define the problem	Actors, system boundary, success criteria, functional requirements	Prevent building the wrong system
2. Rank constraints	Correctness, security, latency, availability, throughput, cost	Architectural choices depend on priorities
3. Scale and assumptions	Requests/sec, peak factors, read/write ratio, storage, retention; label unknowns	Quantifies which optimizations are necessary
4. Interfaces	APIs, commands/events, error semantics, idempotency contracts	Establishes behavior before components
5. State and data	Entities, source of truth, key constraints, consistency	Financial errors usually start here
6. Minimum viable architecture	Services, database, synchronous path, external systems	Avoid needless complexity
7. Derive design choices	Alternatives, chosen option, why alternatives lose	Demonstrates architecture judgment
8. Walk critical flows	Success, retry, duplicate, timeout, failure, recovery	Exposes hidden state transitions
9. Scale and protect	Partitioning, cache, rate limits, backpressure, observability	Add only where justified
10. Enterprise review	Security, compliance, migration, ownership, DR, cost	Makes it an operational solution
11. Final diagram and trade-offs	ASCII architecture and explicit limitations	Communicates the complete design

Example: deriving a payment API

Problem: A merchant calls an API to authorize a payment. The merchant may retry after a timeout, and processing depends on an external payment network.

1. **Requirement — one logical operation:** A repeated client request must not create a second financial operation. **Choice:** accept an idempotency key; bind it to a request fingerprint and persist an operation record. **Why:** networks can time out after the server has begun work.
2. **Requirement — durable state:** Payment state changes need atomic constraints. **Choice:** begin with a relational transactional store. **Why:** uniqueness and state transitions matter more than schemaless flexibility at this stage.
3. **Requirement — prompt decision:** The merchant needs an immediate approve/decline/unknown answer. **Choice:** synchronous processing for the authorization decision. **Why:** an entirely asynchronous response does not satisfy that interface.
4. **Requirement — decoupled side effects:** Reporting and notifications must not increase payment latency. **Choice:** publish events after committing state, using a transactional outbox. **Why:** prevents a database commit succeeding while an independent event publish fails silently.
5. **Requirement — ambiguous provider result:** A timeout is not a decline. **Choice:** represent `UNKNOWN/PENDING`, query or reconcile external status, and only retry according to provider idempotency rules. **Why:** blind retries may duplicate charges.
6. **Requirement — sensitive data:** Minimize raw card data propagation. **Choice:** tokenization, tight access controls, encrypted transport and storage, audit logging. **Why:** reduce exposure and compliance scope.



Concepts introduced only when needed: idempotency (same logical request yields no additional side effect); ACID transaction (atomic, consistent, isolated and durable database work); outbox (publish an event from durable committed state); reconciliation (compare independent records to detect and resolve mismatches).

3. Module 1 — Architecture Foundations

These are **20 connected decisions**, not 20 detached lectures. Each one will be revisited in a case study.

Topic	Learn to reason about	First useful case
1. Functional vs. quality requirements	What the system does vs. how well it must do it	Payment authorization
2. Back-of-envelope capacity	Average/peak RPS, data volume, bottlenecks	URL shortener
3. System boundaries	Who owns state, where trust changes	Payment gateway
4. API contracts	REST/gRPC/events, versioning, errors, retries	API gateway
5. Data modeling	Constraints, indexes, keys and state machines	Ledger
6. Relational vs. NoSQL	Transactions, queries, partitioning, flexibility	Ledger vs. feed
7. Consistency	Strong, eventual; stale reads; read-your-writes	Booking
8. Replication	Read scaling, availability, failover lag	Multi-region reads
9. Partitioning/sharding	Capacity, hot keys, routing, rebalancing	Event pipeline
10. Caching	Hit rate, invalidation, TTL, stampede	News feed
11. Message queues	Worker decoupling, retries, dead-letter queues	Notifications
12. Event streams	Ordering, partition keys, replay, consumer groups	Fraud stream
13. Distributed transactions	ACID locality, Saga, compensations	Checkout
14. Idempotency	Retry without extra business effect	Payments
15. Concurrency	Race conditions, optimistic/pessimistic locking	Ticketing
16. Backpressure	Bound load when downstream systems slow	Ingestion
17. Resilience	Timeouts, retries, circuit breakers, bulkheads	Payment gateway
18. Observability	Logs, metrics, traces, business invariants	Authorization
19. Security by design	Authentication, authorization, threat boundaries	IAM/token vault
20. Operational architecture	SLOs, DR, rollout, runbooks and cost	Active-passive DR

Foundational rules of thumb — with limitations

- **Do not default to microservices.** Start with cohesive boundaries; split when independent ownership, scaling or deployment justifies it.
- **Do not cache correctness-critical balances as the source of truth.** A cache can accelerate reads but needs an explicit freshness contract.
- **Do not assume "exactly once" across arbitrary external systems.** Combine at-least-once delivery with idempotent business effects and reconciliation where needed.
- **Do not treat network timeout as operation failure.** Model ambiguous outcomes explicitly.
- **Do not introduce multi-region writes casually.** Consistency and conflict resolution can dominate the design.
- **Do not name a database before describing the access patterns and invariants.**

4. Module 2 — Payments and Financial Systems

Priority: highest domain depth. Trace authorization, clearing, settlement, dispute and audit as **distinct phases**; do not flatten them into a single database write.

#	Design problem	Why this case matters / critical choices
1	Credit-card authorization	Low-latency synchronous decision; idempotency; uncertain responses; risk checks
2	Merchant payment gateway	API authentication, partner integration, retry contracts, provider failures
3	Capture, clearing and settlement	Different financial phases; batch/event workflows; finality and reconciliation
4	Multi-provider payment orchestration	Routing, circuit breakers, provider-specific semantics, fallback safety
5	Double-entry financial ledger	Debit/credit invariants; immutable entries; transactional posting; auditability
6	Refunds and reversals	State machine, partial refunds, duplicate requests, external status
7	Chargebacks/disputes	Long-running workflow, deadlines, evidence, exception queues
8	Peer-to-peer transfer	Atomic ledger entries, holds, concurrency, recipient identity
9	Recurring billing/subscriptions	Scheduling, retry policy, failed payments, idempotent cycles
10	Real-time fraud detection	Feature freshness, scoring latency, rules + models, fail-open/closed trade-off
11	Limits and velocity controls	Atomic counters, sliding windows, hot accounts, correct risk decisions
12	Merchant onboarding / KYC	Human reviews, third-party checks, long-lived durable orchestration
13	Reconciliation engine	Match ledgers to external files/events; discrepancies; reprocessing
14	Transaction notifications	Outbox, event consumers, delivery retries, user preferences
15	Rewards / loyalty points	Earn/burn events, balances, expiry, reversals and consistency
16	Card tokenization vault	Sensitive data isolation, tokens, HSM/KMS, audit and access boundaries

Cross-cutting payment concepts to introduce in these designs

- **Financial invariants:** e.g., a balanced ledger posting must not partially commit; define the invariant before choosing storage.
- **Idempotency scope:** identify the business action, request fingerprint, retention period and response replay rules.
- **Authorization vs. settlement:** a decision to approve an authorization is not the same as final movement of money.
- **Exactly-once business effect:** messaging may redeliver; deduplication and atomic state transitions protect the actual financial effect.
- **Unknown results:** reconcile with independent sources rather than treating timeouts as failure.
- **PII/cardholder-data boundaries:** classify and minimize data flows; use applicable PCI/security controls.
- **Fraud vs. reliability:** sometimes a failing risk service implies decline, sometimes controlled fallback; derive it from policy and threat model.

5. Module 3 — Scalable Distributed Applications

#	Design problem	Main architectural choice
17	URL shortener	Unique IDs, key-value lookup, redirects, hot-key caching
18	Distributed rate limiter	Token bucket vs. sliding window; atomic distributed counters
19	API gateway	Authentication boundary, routing, rate limits, request policies
20	Multi-channel notifications	Fan-out, queue vs. stream, retries, dead-letter handling
21	Distributed job scheduler	Leader election, claiming work, duplicate-safe execution
22	Webhook delivery platform	Delivery contracts, signed payloads, retry windows, backoff
23	WhatsApp-style chat	WebSocket vs. polling, ordering, offline delivery, fan-out
24	Social media feed	Fan-out on write vs. fan-out on read, celebrity skew
25	Search autocomplete	Prefix index/trie, ranking, freshness, cache
26	File sharing	Object storage, metadata DB, pre-signed URLs, authorization
27	Video streaming	Transcoding jobs, CDN, segments, adaptive bitrate
28	E-commerce checkout	State transitions, reservation, payment, order Saga

Distinct patterns to extract

- **Read-heavy vs. write-heavy:** drives indexing, denormalization, caching and partitioning.
- **Push vs. pull:** for chat, feeds and notifications; connection management costs matter.
- **Fan-out on write vs. read:** precompute for cheap reads or compute on demand for lower write amplification.
- **Queue vs. log:** a work queue distributes tasks; an event log enables independent consumers/replay, with product-specific semantics.
- **Object storage vs. database blobs:** separate large content delivery from searchable metadata and authorization.

6. Module 4 — Distributed Transactions and Workflows

#	Design problem	Main architectural choice
29	Inventory reservation	Atomic stock decrement, TTL reservations, expiry races
30	Ticket booking	Optimistic vs. pessimistic locking; unique seat constraints
31	Driver-to-rider matching	Geospatial index, freshness, claim ownership, concurrency
32	Order fulfillment	Saga orchestration vs. choreography; compensating actions
33	Coupon redemption	Global limits, per-user uniqueness, hot keys
34	Wallet balance	Immutable ledger vs. mutable read model; concurrent spending
35	Shipment tracking	Event ingestion, out-of-order events, eventual consistency
36	Durable workflow engine	State machines, timers, retries, cancellation and recovery
37	Transactional outbox	Atomic data update + durable event intent; relay deduplication
38	Idempotent consumer	At-least-once events, inbox table, replay-safe side effects

Key distinction

A **database transaction** protects changes inside its transactional boundary. A **Saga** coordinates a longer business workflow across systems; compensation is not the same as magically rolling back completed external actions. Explain what must be compensated, what cannot be undone, and how operators recover stuck workflows.

7. Module 5 — Data Architecture and Streaming

#	Design problem	Main architectural choice
39	Event ingestion pipeline	Brokers, partitions, ordering keys, schema evolution
40	Real-time analytics dashboard	Stream aggregations, materialized views, freshness SLA
41	Metrics and log platform	High-cardinality cost, time-series storage, retention
42	Search indexing pipeline	Source DB vs. derived index; update lag and rebuild
43	Change Data Capture	Database log tailing vs. application dual writes
44	Data warehouse ETL/ELT	OLTP vs. OLAP, batch vs. incremental, lineage
45	Immutable audit trail	Tamper resistance, sequence, retention, access controls
46	Customer 360 profile	Identity matching, source-of-truth ownership, privacy
47	Data retention and deletion	Policy propagation, legal holds, backups, audit evidence

Concepts in context

- **OLTP:** short operational reads/writes, often transactional; **OLAP:** large analytical scans and aggregations.
- **CDC:** propagate committed database changes through a log rather than trying to write to two systems in one request.
- **Materialized view:** precomputed read model, with declared staleness and rebuild strategy.
- **Event time vs. processing time:** late/out-of-order events can affect aggregates; watermarks and correction policy matter.
- **Data lineage:** know which source, transformation and policy produced a result.

8. Module 6 — Scalability, Cloud and Reliability

#	Design problem	Main architectural choice
48	Multi-region read service	Replicas, latency routing, stale-read tolerance
49	Active-passive failover	Regional recovery, replicated state, cutover procedure
50	Active-active transactions	Write coordination, conflict resolution, partition tolerance
51	Distributed cache tier	Cache-aside, TTL, consistency, stampede prevention
52	Flash-sale traffic	Admission control, virtual queues, shed load safely
53	Backup and disaster recovery	RPO, RTO, tested restore, data integrity
54	Circuit breaker and fallback	Timeouts, retry budgets, bulkheads, failure isolation
55	Multi-tenant platform	Isolation, noisy neighbors, cost/accounting boundaries
56	Safe deployment pipeline	Canary/blue-green, compatibility, feature flags, rollback

Must understand

- **RPO:** maximum acceptable data loss measured in time. **RTO:** maximum acceptable recovery time.
- **Availability vs. correctness:** degraded reads may be acceptable; incorrect balances or duplicate postings may not be.
- **Backpressure:** reduce accepted work to what downstream systems can process, rather than filling unlimited queues.
- **Retry amplification:** nested automatic retries can turn a partial outage into a total one; use bounded budgets and jitter.
- **Operational readiness:** dashboards, alerts, runbooks, DR tests, ownership and on-call dependencies.

9. Module 7 — Security and Enterprise Architecture

#	Design problem	Main architectural choice
57	Enterprise single sign-on	OIDC/SAML, trust federation, session and token lifetimes
58	Multi-factor authentication	Step-up verification, recovery, device trust, risk signals
59	Authorization platform	RBAC vs. ABAC, policy evaluation, revocation
60	Secrets and key management	KMS/HSM, key lifecycle, rotation, least privilege
61	B2B partner API	Contract versioning, mTLS/OAuth, quotas, failure isolation
62	Mainframe modernization	Strangler migration, anti-corruption layer, dual-run validation
63	Hybrid-cloud integration	Connectivity, trust boundary, egress, latency and ownership
64	Cardholder-data boundary	Tokenization, segmentation, audit and PCI scope
65	Consent and privacy service	Consent versioning, policy checks, deletion/retention

Architecture review lens

For each major design, answer within the explanation:

- **Business alignment:** Which capability, customer journey or regulatory need is served?
- **Integration:** Which systems already exist? What are their contracts and failure modes?
- **Security:** Who is trusted, where does sensitive data move, and who can access it?
- **Ownership:** Which team supports each service, schema, shared component and incident?
- **Delivery:** How can it migrate safely with rollback and backward-compatible interfaces?
- **Cost:** What grows with traffic, storage, redundancy, vendor calls and operational effort?
- **Governance:** How are decisions documented, reviewed and revised as constraints change?

10. Module 8 — Advanced Architecture Breadth

#	Design problem	Main architectural choice
66	Online ML inference	Feature store freshness, serving latency, safe fallback
67	Recommendation system	Candidate retrieval vs. ranking, online/offline features
68	Enterprise RAG assistant	Retrieval permissions, data leakage prevention, grounding
69	Nearby location search	Spatial indices, cell systems, precision vs. cost
70	Collaborative editing	OT vs. CRDT, ordering, offline merge
71	Real-time ad auction	Tight latency budget, scoring, budget enforcement
72	IoT telemetry platform	Device identity, ingestion bursts, ordering, retention
73	Media transcoding	Asynchronous job graph, retries, storage lifecycle

Treatment: Pattern-focused unless a case introduces a genuinely new architecture choice. These extend breadth; they do not replace the financial and enterprise core.

11. Sixteen Full-Depth Anchor Designs

These receive the complete 11-step treatment from the Method section; other cases reuse their patterns and focus on what differs.

Order	Anchor design	Deep concepts it teaches	Related cases made easier
1	Card authorization	Low-latency state machines, idempotency, unknown outcomes	Gateway, refunds, limits
2	Double-entry ledger	Atomicity, immutable postings, invariants	Wallet, transfer, rewards
3	Settlement + reconciliation	Async phases, files/events, mismatches, reprocessing	Billing, disputes, accounting
4	Fraud decisioning	Feature freshness, stream processing, inference fallback	Velocity limits, real-time analytics
5	Payment gateway	Partner contracts, retries, resilience, security boundary	B2B API, webhooks
6	Distributed rate limiter	Atomic counters, windows, hot keys	Quotas, coupons, risk limits
7	E-commerce checkout	Saga and outbox, compensation	Order fulfillment, inventory
8	Ticket booking	High contention, locks, transactions	Reservations, coupon limits
9	Notification service	Queue semantics, worker pools, deduplication	Webhooks, scheduled jobs
10	Chat	WebSockets, ordering, online/offline state	Collaborative editing, live tracking
11	News feed	Fan-out patterns, read/write scaling	Recommendations, activity feeds
12	Search/indexing service	Materialized index, eventual consistency, rebuilds	Autocomplete, customer search
13	Streaming analytics pipeline	Partitions, replay, windows, CDC	Fraud features, metrics
14	Multi-region architecture	Failover, replica lag, RPO/RTO	Active-passive, active-active
15	Enterprise IAM	Identity federation, policies, revocation	B2B access, tenant isolation
16	Legacy modernization	Incremental migration, parallel runs, integration	Hybrid cloud, API modernization

Depth rule

Do not force the same depth on all 73 problems. Spend conceptual effort where one design introduces a new pattern or a non-obvious failure mode. For variations, explain the changed requirement, changed architecture, and changed trade-off without re-teaching shared basics.

12. Architecture Decision Reference

Decision	Usually choose A when...	Usually choose B when...	Hidden trap to discuss
Relational DB vs. NoSQL	Cross-row integrity, relational queries, durable transactions	Data/access patterns justify alternate model or scaling	A NoSQL name alone does not solve scaling
Monolith vs. microservices	Tight coupling, small ownership boundary, deployment simplicity	Independent teams/capabilities need separate deployment	Distributed operations cost
REST vs. gRPC	External/public contracts, broad interoperability	Internal typed RPC and efficiency matter	Versioning and observability
Request/response vs. events	Caller needs an immediate answer	Work can complete independently or be replayed	False assumption of immediate consistency
Work queue vs. event log	Tasks should be processed by workers	Multiple consumers need durable event history	Ordering/replay semantics differ
Strong vs. eventual consistency	Correctness needs latest state now	Staleness has explicit safe bounds	User-visible invariants
ACID transaction vs. Saga	All updates share transaction boundary	Multiple independently committed systems	Compensation is not full rollback
Optimistic vs. pessimistic lock	Conflicts are infrequent	Contention is high, blocking acceptable	Deadlocks and retries
Cache-aside vs. write-through	App controls missing/read behavior	Fresh cache writes are worth write coupling	Invalidation and drift
Replication vs. sharding	Availability/read scaling is goal	Write/storage capacity exceeds one shard	Hot keys and cross-shard queries
Normalized vs. denormalized	Write integrity and canonical data	Read pattern warrants derived projections	Update lag and recomputation
CQRS vs. unified model	Read/write workloads diverge sharply	Simpler model meets requirements	Projection rebuild and drift
Polling vs. WebSocket	Updates are infrequent and latency loose	Frequent low-latency bidirectional updates	Connection load and reconnection
Single-region vs. multi-region	Simpler consistency and sufficient DR	Availability/latency/data policy justify regions	Cross-region data correctness
Active-passive vs. active-active	Simpler failover with accepted RTO	Continuous regional serving is essential	Write conflicts and split brain
Orchestration vs. choreography	Central visibility and business process control	Loosely coupled independent event reactions	Hidden workflow dependencies
Fail-open vs. fail-closed	Degradation is safe and policy permits	Denying unsafe operations is required	Risk must be explicitly owned
Managed vs. self-hosted	Reduced operational burden is valuable	Specific control/constraints justify ownership	Exit, cost, and support model
Batch vs. streaming	Freshness is not urgent, throughput matters	Low-latency updates create business value	Event time and late data
Build vs. buy	Differentiation, control or integration justify build	Mature capability can be procured safely	Vendor SLA, portability, lock-in

Decision explanation pattern: State the constraint -> identify at least two credible options -> choose one -> name the cost or failure mode it introduces -> define when to revisit it.

13. Learning Order — Dependency-Driven, Not Timed

The study sequence follows concept dependencies. No clocks, exercises or assessments are included.

Pass A — Establish the minimum vocabulary

Cover Module 1 while immediately illustrating each concept with payment authorization, URL shortener, ticket booking, notifications and a ledger. Do not defer practice designs until after a large theory chapter.

Pass B — Financial correctness core

Walk through **card authorization -> ledger -> settlement/reconciliation -> fraud decisioning -> payment gateway**. These are related but have different latency, consistency, failure and data-retention constraints.

Pass C — General distributed patterns

Use **rate limiter -> notification service -> checkout/Saga -> ticket booking -> chat -> feed -> search**. This covers atomic counters, work queues, workflows, concurrency, persistent connections, fan-out and indexes.

Pass D — Data, scale and reliability

Use **streaming analytics -> CDC -> multi-region and disaster recovery -> caching/failover**. Contrast correctness-preserving write paths with eventually consistent projections.

Pass E — Solution Architect lens

Use **IAM -> partner integrations -> legacy modernization -> hybrid cloud**. Cover architecture governance, cloud/on-prem boundaries, incremental migration, operating model, security and cost.

Pass F — Complete breadth

Process the remaining case catalog as targeted variations. Prioritize new patterns, not repetition of technologies already understood.

Material deliberately excluded

- Timed schedules and calendar allocations.
- Quizzes, mock interviews, assessments or scorecards.
- Product-name memorization without underlying trade-off reasoning.
- Unsupported claims about American Express's internal interview questions or private architecture.

14. Reusable Case-Study Template

Copy this structure when we begin any problem:

1. **Problem statement:** User journey, actors and business purpose.
2. **Functional requirements:** Essential operations; explicitly excluded scope.
3. **Quality requirements:** Priority-order correctness, security, latency, reliability, throughput, cost.
4. **Assumptions and rough capacity:** Numbers only when grounded or explicitly illustrative.
5. **Interfaces:** Endpoints, events, auth, idempotency and error semantics.
6. **Data and invariants:** Schemas, unique constraints, source of truth, lifecycle.
7. **Baseline ASCII diagram:** Minimal architecture and request flow.
8. **Decision-by-decision derivation:** Requirement -> options -> choice -> why -> trade-off.
9. **Critical sequences:** Success, duplicate, timeout, partial failure, recovery.
10. **Scale, resilience and security:** Bottlenecks, monitoring, rollback, operational playbooks.
11. **Final architecture:** Consolidated ASCII diagram and remaining risks.
12. **What changes if constraints change:** A small set of meaningful variations, with no assessment questions.

Explanation style

- Define **CQRS**, **Saga**, **CDC**, **outbox**, **sharding**, **replication**, **tokenization**, **HSM**, **RTO/RPO**, etc. in one or two sentences **where first needed**.
- Prefer **ASCII diagrams** over graphics.
- Highlight **the invariant or requirement** before naming technology.
- Make uncertain assumptions explicit; never present illustrative capacity estimates as company facts.
- Include operational and enterprise implications, not just API and database choices.

15. References and Boundaries

The preparation roadmap is a practical synthesis of generally applicable architecture patterns, not a representation of American Express internal systems or a verified interview question bank.

Official references to consult when a specific concept is taught:

- **AWS Well-Architected Framework:** https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html — architecture quality, security, reliability, performance, cost and operations.
- **NIST SP 800-207, Zero Trust Architecture:** https://csrc.nist.gov/pubs/sp/800/207/final — trust boundaries and access principles.
- **PCI Security Standards Council Document Library:** https://www.pcisecuritystandards.org/document_library/ — current PCI DSS materials; verify applicable standard/version before asserting requirements.
- **American Express Careers:** https://www.americanexpress.com/en-us/careers/ — public role context, not a predictor of a specific interview.

PDF/export note: The accompanying offline converter generates the same tabbed HTML reader from this Markdown. Its **Export PDF** action opens the browser print dialog; select **Save as PDF**. No cloud service or external libraries are required. Browsers control the final PDF filename and print settings.